Advanced Data Structures and Algorithm Analysis

Laboratory Project Presentation · Project 1

Performance Comparison and Analysis of Search Trees

Yixiang Chen **Runhui Lin**

School of Mathematical Sciences, Zhejiang University

Fall–Winter 2026

Presentation Outline

Part I: Specification & Design

- **Introduction**
  - Five Search Tree Families
  - Three Benchmark Workloads ($N \leq 50{,}000$)
- **Data Structure & Algorithm Specification**
  - Node Layouts & Iterative Unbalanced BST
  - AVL Tree, Splay Tree, Red-Black Tree, B^+ Tree

Part II: Experiments & Analysis

- **Testing Results**
  - Functional Verification & Graphviz Topologies
  - Benchmark Timings & Runtime Curves
- **Analysis and Comments**
  - Time & Space Complexity Comparison
  - Three Key Empirical Findings & Optimizations

SECTION 1

Introduction

1. Introduction
   - Search Tree Families and Benchmark Workloads
2. Data Structure and Algorithm Specification
3. Testing Results
4. Analysis and Comments

Problem Statement & Five Search Tree Families

- **Goal:** Implement, verify, and benchmark **5 search tree structures** in C across $N \in [1{,}000, 50{,}000]$ distinct integer keys.

Five Candidate Structures

- **Unbalanced BST:** Baseline binary tree; $O(\log N)$ average, $\Theta(N)$ worst-case chain.
- **AVL Tree:** Strict height balance $|\mathrm{BF}(v)| \leq 1$ via single/double rotations.
- **Splay Tree:** Self-adjusting; moves accessed keys to the root ($O(\log N)$ amortized).
- **Red-Black Tree:** Two-color path invariant ($H \leq 2\log_2(N+1)$, $O(1)$ rotations per update).
- **Order-4 B^+ Tree:** Multi-way routing internal nodes + singly linked data leaves.

Three Canonical Benchmark Workloads

Scenario 1: Ordered + Same

- **Insert:** $0, 1, \ldots, N-1$
- **Delete:** $0, 1, \ldots, N-1$
- **Focus:** Monotone insertion & root-side deletion.

Scenario 2: Ordered + Reverse

- **Insert:** $0, 1, \ldots, N-1$
- **Delete:** $N-1, \ldots, 1, 0$
- **Focus:** Deepest-leaf deletion on skewed trees.

Scenario 3: Random + Random

- **Insert:** Uniform random shuffle
- **Delete:** Independent shuffle
- **Focus:** Average-case $\Theta(N \log N)$ constant factors.

Evaluation Protocol: $N = 1{,}000, 2{,}000, \ldots, 50{,}000$ (50 grid points), averaged over 3 independent trials (`clock()`).

SECTION 2

Data Structure and Algorithm Specification

1 Introduction

2 Data Structure and Algorithm Specification
- Node Layouts and Unbalanced BST
- AVL Tree and Splay Tree
- Red-Black Tree and Order-4 B+ Tree

3 Testing Results

4 Analysis and Comments

Overview of Node Specifications & Iterative BST

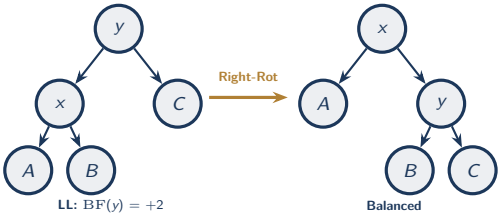
Structure	Node Fields	Core Invariant
Unbalanced BST	`val`, `Left`, `Right`	Standard BST ordering; no balance metadata
AVL Tree	`val`, `height`, `BF`, `Left`, `Right`	$\lvert \mathrm{BF}(v) \rvert = \lvert h(v.\mathrm{Left}) - h(v.\mathrm{Right}) \rvert \leq 1$
Splay Tree	`key`, `left`, `right`	Accessed / updated node splayed to root
Red-Black Tree	`key`, `color`, `left`, `right`, `parent`	Black root, no red–red edge, equal black-height
Order-4 B$^+$ Tree	`isLeaf`, `numKeys`, `keys[]`, `children[]`, `parent`, `next`	Internal: 2–4 children; Leaf: 2–3 keys + linked list

Unbalanced BST (`binaray_search_tree.h`): Iterative Implementation

- Recursive BST overflows the call stack at $N \approx 26{,}000$ under sorted insertion (depth N).
- Iterative `parent`/`current` pointers reduce auxiliary stack space to $O(1)$.

AVL Tree (`AVL_tree.h`): Height Balance & Rotations

- **Balance Factor:** $\mathrm{BF}(v) = h(v.\mathrm{left}) - h(v.\mathrm{right})$
- **Four Rotation Types (**$|\mathrm{BF}(v)| = 2$**):**
  - **LL / RR:** Single rotation
  - **LR / RL:** Double rotation



Algorithm: Rebalance-AVL(v)

```
1.  Update h(v) and BF(v) ← h(v.left) − h(v.right).
2.  if BF(v) > 1 then                                   ▷ Left-heavy
3.    if BF(v.left) < 0 then v.left ← Left-Rotate(v.left)
4.    return Right-Rotate(v)
5.  if BF(v) < −1 then                                  ▷ Right-heavy
6.    if BF(v.right) > 0 then
      v.right ← Right-Rotate(v.right)
7.    return Left-Rotate(v)
8.  return v
```

Splay Tree (`splay_tree.h`): Splaying, Split & Join

- **Three Splaying Rotation Types** (move accessed node x to root):

  Zig (at root) **Zig-Zig** (same-side: rotate g then p) **Zig-Zag** (opposite-side: rotate p then g)

Split-Based $\mathrm{Splay\text{-}Insert}(r, k)$

1. $r \leftarrow \mathrm{Splay}(r, k)$; if $r.\mathrm{key} = k$, return r.
2. Allocate new root z with key k.
3. Split r around k into $z.\mathrm{left}$ and $z.\mathrm{right}$.

Join-Based $\mathrm{Splay\text{-}Delete}(r, k)$

1. $r \leftarrow \mathrm{Splay}(r, k)$; if $r.\mathrm{key} \neq k$, return r.
2. Splay max of $r.\mathrm{left}$: $r' \leftarrow \mathrm{Splay}(r.\mathrm{left}, k)$.
3. Join $r'.\mathrm{right} \leftarrow r.\mathrm{right}$ and free r.

Red-Black Tree (`red_black_tree.h`): Fix-up Cases

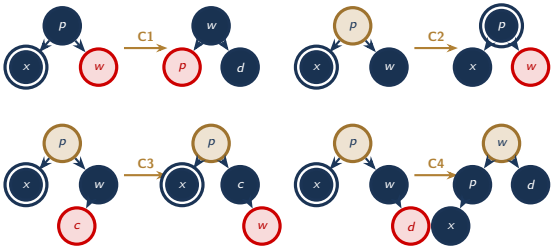
Insertion Fix-up (≤ 2 Rotations)

- **Case 1 (Red uncle u):** Recolor p, u, g; move up.
- **Case 2 (Black u, inner):** Rotate $p \rightarrow$ Case 3.
- **Case 3 (Black u, outer):** Recolor & rotate g.

Deletion Fix-up (≤ 3 Rotations)

- **Case 1:** Red sibling $w \rightarrow$ rotate p.
- **Case 2:** Black w & black nephews $\rightarrow$ recolor.
- **Case 3:** Red inner nephew $\rightarrow$ rotate w.
- **Case 4:** Red outer nephew $\rightarrow$ rotate p, done.

Four Double-Black Deletion Cases



Order-4 B^+ Tree (`B_plus_tree.h`): Split, Borrow & Merge

- **Structure ($M = 4$):** All data keys reside in linked leaves (2–3 keys); internal nodes store 1–3 routing separators $\mathrm{key}[i] = \min(\mathrm{child}[i+1])$.

Insertion: Overflow Split

- **Leaf Overflow (4 keys):** Split into $2 + 2$ keys and link via `next`.
- **Internal Overflow (5 children):** Split into $3 + 2$ children and propagate upward.

Deletion: Underflow Borrow / Merge

- **Sibling Borrow:** Transfer boundary key/child if adjacent sibling has > 2 keys.
- **Sibling Merge:** Merge underfull node with sibling and pull parent separator.

SECTION 3

Testing Results

1 Introduction

2 Data Structure and Algorithm Specification

3 Testing Results
- Functional Verification and Topologies
- Benchmark Timings and Curves

4 Analysis and Comments

Test Suite & Structural Topology Verification



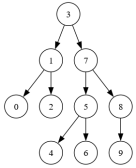
(a) BST

Right chain



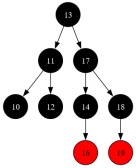
(b) Splay

Left chain



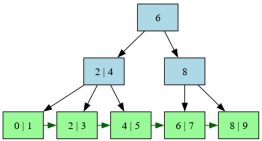
(c) AVL Tree

Height-balanced (0..9)



(d) Red-Black Tree

Keys 10..19, delete 15

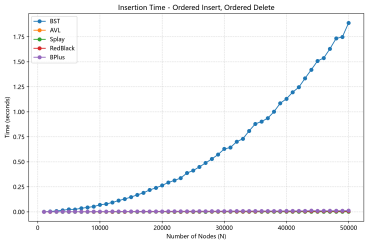


(e) Order-4 B$^+$ Tree: Internal separator nodes (blue) and linked data leaves (green)

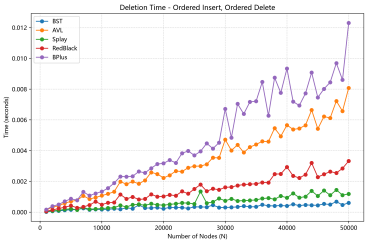
Execution Time Summary Table (in Milliseconds)

Scenario	Tree	$N = 1{,}000$	$N = 10{,}000$	$N = 20{,}000$	$N = 30{,}000$	$N = 40{,}000$	$N = 50{,}000$
Scenario 1: Ordered Insertion (ms)	BST	0.754	69.678	264.054	628.587	1129.765	**1888.738**
	AVL	0.183	1.702	2.992	4.424	6.619	7.833
	Splay	0.036	0.250	0.581	0.514	0.736	**0.778**
	Red-Black	0.136	1.210	2.590	3.873	5.615	6.942
	B^+ Tree	0.247	2.972	4.557	6.809	8.891	12.355
Scenario 1: Ordered Deletion (ms)	BST	0.016	0.171	0.217	0.301	0.399	**0.599**
	AVL	0.130	1.069	2.221	4.712	5.656	8.075
	Splay	0.044	0.227	0.449	0.735	0.929	1.177
	Red-Black	0.054	0.485	1.021	1.609	2.931	3.319
	B^+ Tree	0.159	1.332	3.167	6.714	9.338	12.306
Scenario 2: Reverse Deletion (ms)	BST	0.684	63.698	251.397	590.257	1139.824	**1777.648**
	AVL	0.121	1.130	2.412	4.375	5.891	8.341
	Splay	0.018	0.110	0.227	0.336	0.391	**0.646**
	Red-Black	0.071	0.522	1.059	2.438	2.246	3.177
	B^+ Tree	0.157	1.281	3.182	5.975	6.615	8.095
Scenario 3: Random Insertion (ms)	BST	0.090	0.843	1.941	3.838	4.015	**5.524**
	AVL	0.334	2.155	4.478	9.397	10.925	12.785
	Splay	0.166	1.858	3.941	7.184	8.619	10.446
	Red-Black	0.179	1.588	3.563	5.608	7.387	10.409
	B^+ Tree	0.276	3.000	7.625	12.549	16.264	19.595
Scenario 3: Random Deletion (ms)	BST	0.074	0.764	1.753	2.985	4.417	**5.359**
	AVL	0.276	1.851	4.748	7.178	10.779	13.538
	Splay	0.198	1.371	3.342	6.337	7.656	10.079
	Red-Black	0.239	1.095	2.456	4.473	5.650	7.303
	B^+ Tree	0.235	2.548	6.142	11.828	16.430	20.871

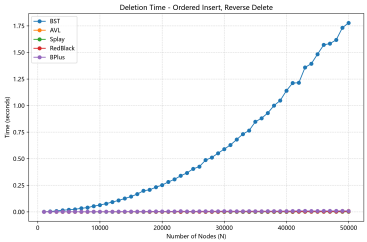
Runtime Curves: Scenario 1 & Scenario 2 (Monotone Inputs)



(a) Ordered Insert



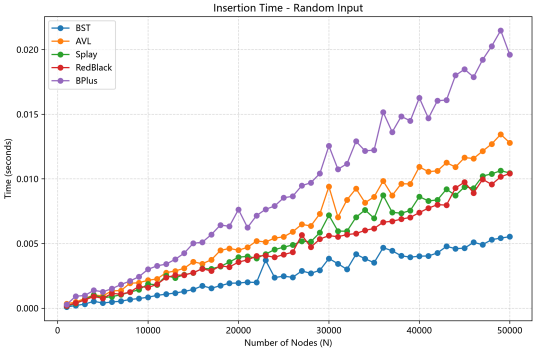
(b) Scenario 1: Same-Order Delete



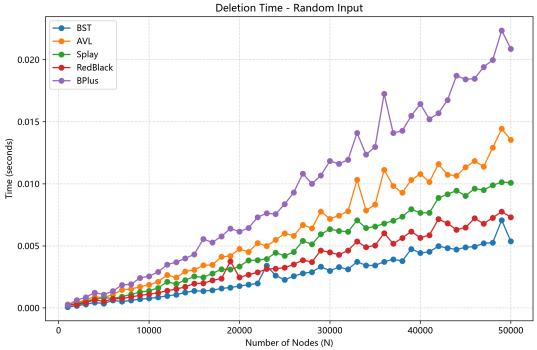
(c) Scenario 2: Reverse Delete

- **Unbalanced BST:** $\Theta(N^2)$ on ordered insert (1.89 s) and reverse delete (1.78 s), but $\Theta(N)$ on same-order delete (0.60 ms).
- **Splay Tree:** Fastest on sorted insert (0.78 ms) and reverse delete (0.65 ms).

Runtime Curves: Scenario 3 (Random Insert + Random Delete)



(a) Random Insertion Time



(b) Random Deletion Time

- All five trees scale as $\Theta(N \log N)$ under random inputs (< 21 ms at $N = 50{,}000$).
- Ranking: **BST** $<$ **Red-Black** $<$ **Splay** $<$ **AVL** $<$ **B$^+$ Tree**.

SECTION 4

Analysis and Comments

1 Introduction

2 Data Structure and Algorithm Specification

3 Testing Results

4 Analysis and Comments
- Time and Space Complexity
- Key Empirical Findings
- Further Improvements and Summary

Asymptotic Time & Space Complexity Comparison

Structure	Avg Op	Worst Op	Scenario 1	Scenario 2	Node Size / Aux Stack
Unbalanced BST	$\Theta(\log N)$	$\Theta(N)$	$\Theta(N^2)$ / $\Theta(N)$	$\Theta(N^2)$ / $\Theta(N^2)$	24 B, $O(1)$ iterative
AVL Tree	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$	32 B, $O(\log N)$ stack
Splay Tree	$\Theta(\log N)$ amort.	$\Theta(N)$	$\Theta(N)$ / $\Theta(N)$	$\Theta(N)$ / $\Theta(N)$	24 B, $O(H)$ stack
Red-Black Tree	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$	40 B, $O(1)$ iterative
Order-4 B$^+$ Tree	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$	Dynamic arrays ($M = 4$)

Memory Layout Highlights (64-bit Target)

- **Binary Trees:** 24–40 bytes/node depending on balance metadata (`height`/`BF` in AVL; `color`/`parent` in RB).
- **Order-4 B$^+$ Tree:** Each node allocates dynamic `keys[4]` and `children[5]` arrays, reducing tree depth to $\log_{2\ldots4} N$ at the cost of extra heap allocations.

Finding 1: Asymmetry of Unbalanced BST Deletion

Why Same-Order Delete Takes 0.60 ms While Reverse Delete Takes 1.78 s?

After inserting $0, 1, \ldots, N-1$, the BST is a right-skewed chain $0 \to 1 \to \cdots \to N-1$:

- **Ordered Insertion ($\Theta(N^2)$, 1.89 s):** The k-th insertion walks k nodes to the bottom leaf:

$$\sum_{k=0}^{N-1} k = \frac{N(N-1)}{2} = \Theta(N^2) \quad (N \times 5 \implies 27.1\times \text{ runtime}).$$

- **Scenario 1 — Increasing Delete ($\Theta(N)$, 0.60 ms):** Key $0, 1, \ldots$ is always the **current root**, removed in $O(1)$ time via `root` $\leftarrow$ `root->Right`.
- **Scenario 2 — Reverse Delete ($\Theta(N^2)$, 1.78 s):** Key $N-1, N-2, \ldots$ is always the **deepest leaf**, requiring $N-k$ steps from the root on every deletion.

Finding 2: $\Theta(N)$ Sequential Locality of Splay Trees

Why Splay Tree Dominates Monotone Workloads (< 1.2 ms at $N = 50{,}000$)

- **Increasing Insertion ($\Theta(N)$, 0.78 ms):** Each inserted key k becomes the new root with an empty right subtree $\Longrightarrow$ inserting $k+1$ terminates at the root in $O(1)$ time, forming a left-skewed spine $N-1 \to \cdots \to 0$.
- **Reverse Deletion ($\Theta(N)$, 0.65 ms):** Deleting $N-1, N-2, \ldots, 0$ repeatedly removes the current root of the left spine in $O(1)$ time.
- **Increasing Deletion ($\Theta(N)$ amortized, 1.18 ms):** The first `findSplay(root, 0)` performs *zig-zig* rotations along the spine, **halving the depth of every node** and folding the chain into a shallow tree.

Finding 3: Trade-offs Under Random Workloads

At $N = 50{,}000$, uniform random updates yield:

BST (5.5 ms) < **Red-Black** (10.4/7.3 ms) < **Splay** (10.4/10.1 ms) < **AVL** (12.8/13.5 ms) < **B**$^+$ (19.6/20.9 ms)

- **Unbalanced BST:** Expected depth $2 \ln N \approx 1.39 \log_2 N$ with zero rotation/metadata overhead.
- **Red-Black vs. AVL Tree:** Red-Black is 20%–45% **faster** (≤ 2 insert / ≤ 3 delete rotations vs. up to $\Theta(\log N)$ AVL deletion rotations).
- **Splay Tree:** Loses sequential locality under uniform random access; pays continuous splaying overhead.
- **Order-4 B$^+$ Tree:** Highest CPU overhead in RAM due to small-node array shifting, `malloc` calls, and `refreshBPlusAncestors` traversal.

Further Improvements & Practical Takeaways

Further Possible Engineering Improvements

1. **Cache Minimum Key in B^+ Nodes:** Avoid `refreshBPlusAncestors` walking down `children[0]` on every update.
2. **Fixed-Size Arrays in B^+ Struct:** Embed `int keys[M]` and `BPlusTreeNode* children[M+1]` directly to eliminate extra `malloc` calls.
3. **Top-Down Splaying & Memory Pool:** Use iterative top-down splaying and a pre-allocated node memory pool (**from OOP Lab 7 last semester**).

Summary: Choosing the Right Search Tree

- **Purely Random Keys:** Unbalanced BST (fastest constant factor, no worst-case protection).
- **Skewed / Sequential Locality:** Splay Tree ($\Theta(N)$ sequential performance).
- **General-Purpose Dictionary:** Red-Black Tree (best worst-case guarantee vs. rotation cost).

Thank You for Listening!

Questions & Discussion

Yixiang Chen **Runhui Lin**

School of Mathematical Sciences, Zhejiang University

Advanced Data Structures and Algorithm Analysis · Fall–Winter 2026